# *CSS*

*Cascading Style Sheets*



## CSS

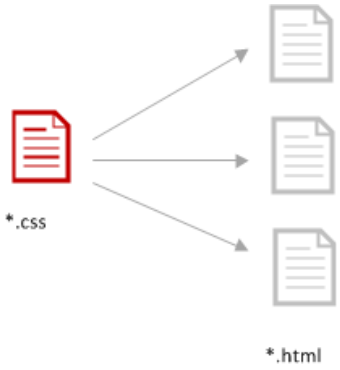
- Incremento da qualidade e consistência gráfica das aplicações
- Separação do **design e do conteúdo**:
  - **HTML** define a estrutura (conteúdo)
  - **CSS** define a formatação/design
    - Layouts
    - Posicionamentos
    - Cores
    - Texto
    - Formulários
    - Tabelas
    - Listas
    - …

*



## CSS

- Vantagens
  - Possibilidade de aplicação imediata a vários documentos HTML
    - Garante que diversos documentos HTML são submetidos às mesmas regras de formatação.
  - Muito maior flexibilidade
    - Alterações centralizadas
    - Evita a repetição de formatações
  - Maior correção
    - Melhora a portabilidade
    - Reduz a possibilidade de erros
    - Maior uniformidade
      - Melhora a consistência gráfica






## Sintaxe CSS

- Composta por:
  - Seletor
  - Declaração /conjunto de declarações
    - Propriedade
    - Valor

*seletor { propriedade:valor;}*

- Cada regra seleciona um elemento/conjunto de elementos e declara toda a formatação associada
- A sintaxe das CSS é **case sensitive**

# Sintaxe CSS

- **Seletor**
  - Elemento crucial na definição de uma regra CSS
    - A compreensão do funcionamento dos seletores é absolutamente essencial para permitir uma correta implementação de CSS
  - Permite identificar os elementos HTML a que a regra CSS se aplica



```
body{ background-color:#FFF;}
```



# Sintaxe CSS

- **Declaração**
  - Uma atribuição de um valor a uma propriedade. Uma regra pode conter várias declarações *(declaration block).*
    - Propriedade
      - Atributo da folha de estilo ao qual deve ser atribuído um valor (ex: *color, …*)

```
body{ background-color:#FFF;}
```

    - Valor
      - Especificação concreta da variável (ex: arial, #0000FF)

```
body{ background-color:#FFF;}
```

# Sintaxe CSS

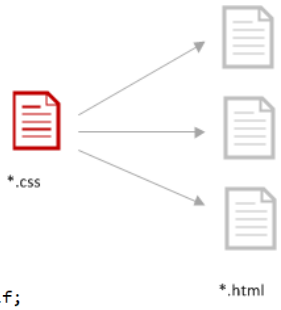
| Sintaxe | Exemplo |
|---|---|
| selector { propriedade: valor} | body {background-color: #FFFFFF} |
| *bloco de declarações ;*<br>selector {<br>propriedade: valor;<br>propriedade: valor;<br>propriedade: valor;<br>} | p { text-align:center;<br>color:red;<br>font-family: arial;} |
| *seleção múltipla ,*<br>selector1, selector2, selector3<br>{ propriedade: valor;} | h1, h2, h3{ color:red;} |



# Ligação CSS / HTML

# Ligação CSS / HTML

- Como aplicar estilos (regras CSS) aos documentos HTML?
  - ***External Style Sheet***
    - Ficheiro externo de extensão *.css que contém as regras CSS
    - Pode ser aplicado a diversos documentos HTML
    - Forma mais poderosa e flexível de incorporar CSS




*styles.css*

```
body{
    height:100%;
    font-size:1rem;
    font-family: 'Montserrat', sans-serif;
}
```

*index.html*

```
<link rel="stylesheet" href="css/styles.css" >
```



# Ligação CSS / HTML

- ***External Style Sheet*** (*.css)
  - Um ficheiro CSS (*.css) é <u>exclusivamente</u> constituído por regras CSS
  - O código CSS <u>não contém tags</u>

seletor{propriedade:valor;}

```
/* style1.css */

p { font-weight:bold;
    color:#F00;}
```

# Ligação CSS / HTML

- ***<link … />***
  - Atributo ***href*** indica a localização do ficheiro *.css . Atributo obrigatório!
  - Atributo ***rel*** define a relação do documento HTML com o ficheiro externo, neste caso é *stylesheet.* Atributo obrigatório!
  - Atributo ***type*** é opcional no HTML5. Em versões anteriores (X)HTML é obrigatório.

```
<!DOCTYPE HTML>
<html>
<head>
<meta charset="utf-8" />
<title> External Style Sheet</title>

<link href="style1.css" rel="stylesheet" type="text/css"/>

</head>

<body>
</body>
</html>
```

```
/* style1.css */

p { font-weight:bold;
    color:#F00;}
```



# Ligação CSS / HTML

- ***External Style Sheet***
  - através de ***@import*** e da tag ***<style>***

```
<!DOCTYPE HTML>
<html>
<head>
    <style>
        @import url("style1.css");
        ...

    </style>
    <meta charset="utf-8" />
    <title> External Style Sheet</title>
</head>

<body>
</body>
</html>
```

```
/* style1.css */

p { font-weight:bold;
    color:#F00;}
```

  - as duas formas de estabelecer a ligação a um ficheiro externo são suportadas pela maioria dos browsers originando resultados semelhantes, no entanto:
    - A tag ***<link>*** é a forma mais eficaz, aquela que assegura melhor performance, para efetuar a ligação de uma HTML a um CSS externo

http://www.stevesouders.com/

# Ligação CSS / HTML

- ***Embedded Style Sheet***
  - Necessita de ser definido em todos os documentos *.html
    - Menos flexível e eficaz do que a ligação a um ficheiro externo
    - Uma alteração na formatação implica uma alteração em cada um dos ficheiros
      - em MPA pode originar inconsistência na formatação entre diferentes *.html






# Ligação CSS / HTML

- ***<style> … </style>***
  - definido do <head> do *.html
    - Atributo type="text/css" é obrigatório nas versões anteriores ao HTML5.

```
<!DOCTYPE HTML>
<html>
<head>
    <style>
        p { font-weight:bold;
            color:#F00;}
    </style>
    <meta charset="utf-8" />
    <title> Embedded Style Sheet</title>
</head>

<body>
</body>
</html>
```

Janela de código CSS

- ***Inline Style***
  - aplicar o estilo localmente através do atributo **style**
    - É a forma **menos flexível** e **mais propensa** a erros para formatar um elemento HTML
    - Se efetuado de forma estática, favorece a inconsistência na formatação dos conteúdos
    - Implica tantas alterações quantas as definições efetuadas

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
</head>
<body>
    <h1 style="color:orange">Inline Style: A evitar!</h1>
</body>
</html>
```






# Seletores CSS

# Seletor CSS

- Permite identificar os elementos HTML aos quais a regra CSS se aplica

*seletor { propriedade:valor;}*

  - **Diferentes tipos de seletores** de forma a permitir a flexibilidade necessária para selecionar qualquer dos elementos/conjunto de elementos definidos no HTML
  - Os seletores CSS são cruciais para a aplicação da tecnologia CSS, mas também são usados pelo JavaScript / Frameworks JS (ex: React) para aceder a determinado elemento HTML.

Seletores CSS **não se esgotam** na tecnologia CSS!

  - Devem ser definidos tendo sempre em consideração os conflitos de formatação:
    - Muito frequentes sempre que o código CSS ganha alguma escala.



# Seletor CSS

- Principais tipos de seletores:

*Elemento*

*Contexto*

*id*

*class*

*Atributo*

# Seletor CSS

- Seletor de **Elemento**
  - Referência direta ao elemento HTML que se pretende formatar
    - Vantagem: simplicidade
    - Desvantagem: pouco específicos

*Elemento*

```
<style>
    p{border:1px solid blue;}
    h2{color:blue}
</style>
```

  - Elementos Agrupados
    - Referência direta aos elementos HTML que se pretendem formatar, os quais devem ser separados por **vírgula (,)**

```
<style>
    h1,h2{ color: #900;}
</style>
```



# Seletor CSS

- Seletor Universal (*)
  - Permite selecionar todos os elementos
  - A utilização deste selector deve ser cuidadosamente ponderada é aplicado a **todos os elementos**
    - podem ser originadas algumas formatações/efeitos não previstos

```
    <style>
        * {color:orange}
    </style>
</head>

<body>
    <p>p first-child</p>
    <ul>
        <li>first option</li>
        <li>second option</li>
    </ul>
    <p>p last-child</p>
</body>
```

p first-child

- first option
- second option

p last-child

# Seletor CSS

- Seletor de **Contexto**
  - Descendentes
    - A formatação dos elementos depende do contexto
    - Elementos separados por **espaço em branco**

*Contexto*

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>

    <style>
        li em {color:orange;}
    </style>

</head>
<body>
    <ul>
        <li><em>1ª opção</em></li>
        <li><em>2ª opção</em></li>
        <li><em>3ª opção</em></li>
    </ul>
    <p>1º <em>parágrafo</em></p>
    <p>2º <em>parágrafo</em></p>
</body>
</html>
```

- *1ª opção*
- *2ª opção*
- *3ª opção*

1º *parágrafo*

2º *parágrafo*

Podem ser criados vários níveis de dependências

```
<style>
    ul li em {color:orange;}
</style>
```



# Seletor CSS

- Seletores de **Contexto**
  - Casos particulares de seletores de contexto
    - Descendentes diretos (**filhos**) - *child selector* **(>)**

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>

    <style>
        span {color:gray;}
        p > span {color:orange;font-size:1.2em}
    </style>

</head>
<body>
    <p><span>1º <span>parágrafo</span></span></p>
    <p><span>2º <span>parágrafo</span></span></p>
</body>
</html
```

1º parágrafo

2º parágrafo

# Seletor CSS

- Seletores de **Contexto**
  - Elementos Adjacentes ***(Adjacent Sibling Selector)*** **(+)**
    - Adjacente: elemento imediatamente seguinte ao elemento que define o contexto

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        h1 + p {color:orange;font-size:1.2em}
    </style>
</head>
<body>
    <h1 id="p1"> Referência</h1>
    <p><span>1º <span>parágrafo</span></span></p>
    <p><span>2º <span>parágrafo</span></span></p>
</body>
</html>
```






# Seletor CSS

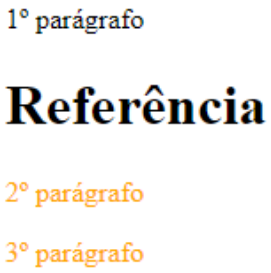
- Seletores de **Contexto**
  - *element1* ~ *element2*
    - seleciona todos os *element2* que são precedidos por um *element1*
      - Os elementos devem ter o mesmo pai
      - O *element2* não tem de ser imediatamente precedido pelo *element1*

```
<!DOCTYPE html>
<html lang="">
<head>
    <meta charset="utf-8">
    <title></title>

    <style>

        h1 ~ p {color:orange}

    </style>
</head>
<body>
    <p> 1º parágrafo </p>
    <h1 id="heading1"> Referência </h1>
    <p> 2º parágrafo </p>
    <p> 3º parágrafo </p>
</body>
</html>
```



# Seletor CSS

- Selectores de ID (#)
  - Atribuir um estilo a  uma única ocorrência de um elemento HTML

*id*

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        p {color:gray;}
       #p1 {color:orange;font-size:1.2em}
    </style>
</head>
<body>
    <p id="p1"> Parágrafo com id=p1</p>
    <p><span>1º <span>parágrafo</span></span></p>
    <p><span>2º <span>parágrafo</span></span></p>
</body>
</html>
```

Parágrafo com id=p1

1º parágrafo

2º parágrafo

  - Os id's são únicos e como tal dispensam a especificação do elemento
    - seletores equivalentes

#p1  p#p1

      - Especificidade diferente



# Seletor CSS

- Seletor de ID (#)
  - Podem ser utilizados como parte de um seletor de contexto

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        li {color:gray;}
        #lista1 li {color:orange;}
    </style>
</head>
<body>
    <ul id="lista1">
        <li>1ª opção / lista id="lista1" </li>
        <li>2ª opção / lista id="lista1" </li>
    </ul>
    <ul>
        <li>1ª opção / lista sem id </li>
        <li>2ª opção / lista sem id </li>
    </ul>
</body>
</html>
```

- 1ª opção / lista id="lista1"
- 2ª opção / lista id="lista1"

- 1ª opção / lista sem id
- 2ª opção / lista sem id

# Seletor CSS

- Seletor de *class* **.**
  - Ao contrário do atributo ID:
    - o atributo *class pode ser partilhada por* múltiplos elementos
  - Um elemento pode ter definida mais de uma *class*
    - Sem limite, separadas por espaço, sendo indiferente a ordem pela qual são definidas.

*class*

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        li {color:gray;}
        .opc {color:orange;}
    </style>
</head>
<body>
    <ul id="lista1">
        <li class="opc">1ª opção / lista id="lista1" </li>
        <li class="opc">2ª opção / lista id="lista1" </li>
    </ul>
    <ul>
        <li class="opc">1ª opção / lista sem id </li>
        <li>2ª opção / lista sem id </li>
    </ul>
</body>
</html>
```

- 1ª opção / lista id="lista1"
- 2ª opção / lista id="lista1"

- 1ª opção / lista sem id
- 2ª opção / lista sem id



# Seletor CSS

- Seletor de *class* **.**
  - *class* aplicada/definida com base num **elemento especifico**

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        li {color:gray;}
        p.opc {color:orange;}
    </style>
</head>
<body>
    <ul id="lista1">
        <li class="opc">1ª opção / lista id="lista1" </li>
        <li class="opc">2ª opção / lista id="lista1" </li>
    </ul>
    <ul>
        <li class="opc">1ª opção / lista sem id </li>
        <li>2ª opção / lista sem id </li>
    </ul>
    <p class="opc">1º Parágrafo</p>
</body>
</html>
```

- 1ª opção / lista id="lista1"
- 2ª opção / lista id="lista1"

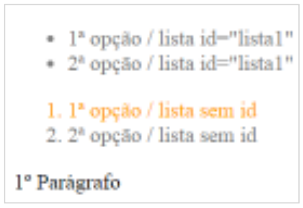
- 1ª opção / lista sem id
- 2ª opção / lista sem id

1º Parágrafo

# Seletor CSS

- Seletor de *class* **.**
  - Podem ser utilizados para criar um seletor contextual
    - São formatados os elementos integrados num outro elemento cujo atributo *class* foi definido com um valor específico.

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        li {color:gray;}
        .lst .opc {color:orange;}
    </style>
</head>
<body>
    <ul id="lista1">
        <li class="opc">1ª opção / lista id="lista1" </li>
        <li class="opc">2ª opção / lista id="lista1" </li>
    </ul>
    <ol class="lst">
        <li class="opc">1ª opção / lista sem id </li>
        <li>2ª opção / lista sem id </li>
    </ol>
    <p class="opc">1º Parágrafo</p>
</body>
</html>
```






# Seletor CSS

- **Importante!**
  - Especificação do elemento e seletor **id**

`h1#d1{color:gray}` Formatação aplicada ao elemento **h1 cujo id é d1**

**≠**

h1 #d1

  - Especificação do elemento e seletor **class**

`h2.c1{color:green}` Formatação aplicada ao elemento **h2 cuja *class* é c1**

**≠**

h2 .c1

O espaço em branco permite implementar seletores totalmente diferentes

# Seletor CSS

```
...
    <style>
        h2.c1{color:orange}
    </style>

</head>

<body>
    <div id="div1" class="c1">
        <h1 id="d1">heading 1</h1>
        <h2 class="c1">heading 2</h2>
        <p class="c1">paragraph1</p>
    </div>
        <h2 class="c1">Other heading 2</h2>
...
```

Formatação aplicada aos elementos h2 cuja **class** é **c1**

heading 1
heading 2
paragraph1
Other heading 2

```
...
    <style>
        .c1 h2{color:orange}
    </style>

</head>

<body>
    <div id="div1" class="c1">
        <h1 id="d1">heading 1</h1>
        <h2 class="c1">heading 2</h2>
        <p class="c1">paragraph1</p>
    </div>
        <h2 class="c1">Other heading 2</h2>
...
```

≠

A ordem dos elementos / espaço em branco é determinante para uma correta implementação de um selector.

Formatação aplicada aos elementos h2 definidos **no contexto** de um outro elemento cuja **class** é **c1**

heading 1
heading 2
paragraph1
Other heading 2



# Seletor CSS

- Seletor de Atributo
  - [attribute]
    - Caso o elemento não seja especificado são afetados todos os elementos que possuem o atributo definido

*Atributo*

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        [id] {color:orange;}
    </style>
</head>
<body>
    <p id="primeiro">1º Parágrafo</p>
    <p> 2º Parágrafo</p>
    <p id="terceiro">3º Parágrafo</p>
    <p> 4º Parágrafo</p>

    <ul id="lista"><li>primeira opção</li></ul>
</body>
</html>
```

1º Parágrafo
2º Parágrafo
3º Parágrafo
4º Parágrafo
- primeira opção

# Seletor CSS

- *Attribute Selectors*
  - Referenciam um elemento através dos seus atributos ou dos valores que assumem
    - ***element [attribute]***
      - referencia os elementos especificados que têm declarado um atributo específico.

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        p[id] {color:orange;}
    </style>
</head>
<body>
    <p id="primeiro">1º Parágrafo</p>
    <p> 2º Parágrafo</p>
    <p id="terceiro">3º Parágrafo</p>
    <p> 4º Parágrafo</p>

    <ul id="lista"><li>primeira opção</li></ul>
</body>
</html>
```

1º Parágrafo

2º Parágrafo

3º Parágrafo

4º Parágrafo

- primeira opção



# Seletor CSS

- ***element [attribute="value"] / [atribute="value"]***
  - referencia os elementos *element* cujo atributo assume um valor *específico*

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        [id="lista"]{color:orange;}
    </style>
</head>
<body>
    <p id="primeiro">1º Parágrafo</p>
    <p> 2º Parágrafo</p>
    <p id="terceiro">3º Parágrafo</p>
    <p> 4º Parágrafo</p>

    <ul id="lista"><li>primeira opção</li></ul>
</body>
</html>
```

1º Parágrafo

2º Parágrafo

3º Parágrafo

4º Parágrafo

- primeira opção

  - Existem algumas variantes de seletor de atributo (menos utilizados) que selecionam os elementos a formatar com base em correspondências parciais dos valores dos atributos.

# Seletor CSS

- *Attribute Selectors*
  - ***element [attribute *= " …"] / [attribute *= "…"]***
    - referencia os elementos cujos valores do atributo especificado contêm uma dada **expressão** (palavra isolada ou inserida em outra palavra)

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        p[id*="paragrafo"] {color:orange;}
    </style>
</head>
<body>
    <p id="paragrafo-primeiro">1º Parágrafo</p>
    <p id="segundo paragrafo">2º Parágrafo</p>
    <p id="paragrafo terceiro">3º Parágrafo</p>
    <p id="quarto">4º Parágrafo</p>
</body>
</html>
```

1º Parágrafo

2º Parágrafo

3º Parágrafo

4º Parágrafo



# Seletor CSS

- *Attribute Selectors*
  - ***element [attribute ^= "…"] / [attribute ^= "…"]***
    - referencia os elementos cujo valor do atributo **se inicia** com uma dada **expressão** (palavra isolada ou inserida em outra palavra)

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        p[id^="paragrafo"] {color:orange;}
    </style>
</head>
<body>
    <p id="paragrafo-primeiro">1º Parágrafo</p>
    <p id="segundo paragrafo">2º Parágrafo</p>
    <p id="paragrafo terceiro">3º Parágrafo</p>
</body>
</html>
```

1º Parágrafo

2º Parágrafo

3º Parágrafo

# Seletor CSS

- *Attribute Selectors*
  - ***element [attribute $=“…”] / [attribute $= “…”]***
    - referencia os elementos cujo valor do atributo **termina com** uma dada **expressão** (palavra isolada ou inserida em outra palavra)

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        p[id$="paragrafo"] {color:orange;}
    </style>
</head>
<body>
    <p id="paragrafo-primeiro">1º Parágrafo</p>
    <p id="segundo paragrafo">2º Parágrafo</p>
    <p id="paragrafo terceiro">3º Parágrafo</p>
</body>
</html>
```

1º Parágrafo

2º Parágrafo

3º Parágrafo

- Lista completa: http://www.w3schools.com/cssref/css_selectors.asp



# Seletor CSS - Exercício

```
<body>
    <p class="cl1"> Parágrafo Exemplo <span> Elemento Inline </span></p>
</body>
```

```
<style>
    p {color:red;}
    span {color:blue;}
    p.span {color:green;}
    p  span {color:green;}
    p  .cl1 {color:orange;}
    p.cl1 {color:orange;}
</style>
```

Parágrafo Exemplo Elemento Inline

Parágrafo Exemplo Elemento Inline

Parágrafo Exemplo Elemento Inline

Parágrafo Exemplo Elemento Inline

Parágrafo Exemplo Elemento Inline

Parágrafo Exemplo Elemento Inline

# Pseudo-Class Selectors



## Pseudo-Class Selector

- Baseados no **estado** do elemento
  - *:link*
    - link não visitado
  - *:visited*
    - link anteriormente visitado
  - *:hover*
    - o cursor do rato encontra-se sobre o elemento
  - *:active*
    - o elemento está a ser selecionado
  - *:focus*
    - elemento selecionado e pronto para receber valores
  - *element:first-child*
    - elemento que é o primeiro filho do seu pai (**excepção!)**

# Pseudo-Class Selector

- *Pseudo-Class selector (exemplo)*
  - *:hover*
    - Sobreposição do cursor do rato

Pseudo-Class: estado link

Pseudo-Class: estado link

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        a:hover {color:white;background-color:#FF6600;font-size:20px;
        }
    </style>
</head>
<body>
    <p>Pseudo-Class: <a href="http://www.isec.pt">estado link</a></p>
</body>
</html>
```



# Pseudo-Class Selector

- *Pseudo-Class selector (exemplo)*
  - *li:first-child*
    - seleciona o elemento que é o primeiro filho do seu pai

```
<!DOCTYPE html>

<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        li:first-child {color:orange;font-size:20px}
    </style>
</head>
<body>
    <ul>
        <li>Primeiro filho de ul</li>
        <li>Segundo filho de ul</li>
        <li>Último filho de ul</li>
    </ul>
    <p> primeiro parárafo </p>
</body>
</html>
```

- Primeiro filho de ul
- Segundo filho de ul
- Último filho de ul

primeiro parárafo

*Seletor **sempre aplicado** no elemento filho*

# Pseudo-Class Selector

- *li:last-child*
  - seleciona o elemento que é o último filho do seu pai

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        li:last-child {color:orange;font-size:20px}
    </style>
</head>
<body>
    <ul>
        <li>Primeiro filho de ul</li>
        <li>Segundo filho de ul</li>
        <li>Último filho de ul</li>
    </ul>
    <p> primeiro parárafo </p>
</body>
</html>
```

- Primeiro filho de ul
- Segundo filho de ul
- Último filho de ul

primeiro parárafo



# Pseudo-Class Selector

- *:first-child ; :last-child*

```
<!DOCTYPE html>
<html lang="">
<head>
    <meta charset="utf-8">
    <title></title>

    <style>
        div:first-child{color:orange}
        div:last-child{color:lightblue}
    </style>
</head>
<body>
    <div> Primeiro elemento div</div>
    <div> Ultimo elemento div</div>
</body>
</html>
```

Primeiro elemento div
Ultimo elemento div

*Alteração da estrutura mantendo o CSS*

```
<body>
    <p>primeiro filho</p>
    <div> Primeiro elemento div</div>
    <div> Ultimo elemento div</div>
    <p>ultimo filho</p>
</body>
```

?

primeiro filho

Primeiro elemento div
Ultimo elemento div

ultimo filho

A formatação não é aplicada, uma vez que a alteração da estrutura originou **que os elementos  deixassem de ser o primeiro e o ultimo filho do seu pai** (neste caso o element <body>)

# Seletor CSS

- *li:nth-child(n)*
  - seleciona o elemento que é o enésimo filho

```
<!DOCTYPE html>

<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        li:nth-child(2) {color:orange;font-size:20px}
    </style>
</head>
<body>
    <ul>
        <li>Primeiro filho de ul</li>
        <li>Segundo filho de ul</li>
        <li>Último filho de ul</li>
    </ul>
    <p> primeiro parárafo </p>
</body>
</html>
```

- Primeiro filho de ul
- Segundo filho de ul
- Último filho de ul

primeiro parárafo



# Pseudo-Class Selector

- *:first-of type ; :last-of-type; :nth-of-type(n)*

```
    <style>
        div:first-of-type{color:orange}
        div:last-of-type{color:lightblue}
    </style>

</head>
<body>
    <p>primeiro filho</p>
    <div>primeiro elemento</div>
    <div>ultimo elemento</div>
    <p>ultimo filho</p>
```

primeiro filho

primeiro elemento
ultimo elemento

ultimo filho

  - Apesar de na maioria das situações produzirem resultados idênticos, os seletores:
    - *:first-of type e :first-child*
    - *:last-of type e :last-child*
    - *:nth-child(n) e :nth-of-type(n)*

**não são** equivalentes!

# Seletor CSS

- Formulários
  - *Pseudo Class Selectors* **específicos** para formulários

| *seletor* | *Exemplo* | *Observações* |
|---|---|---|
| :checked | input:checked | seleciona todos os input checked |
| :disabled | input:disabled | seleciona todos os input que estão inativos |
| :enabled | input:enable | seleciona todos os input que estão ativos |
| :focus | input:focus | seleciona todos os input que "ganham" *focus* |
| :invalid | input:invalid | seleciona todos os input inválidos |
| :optional | input:optional | seleciona todos os input **não** obrigatórios |
| :required | input:required | seleciona todos os input obrigatórios |



# Seletor CSS

- *Pseudo Class Selectors* específicos para formulários




```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        input:disabled { background-color:lightcoral}
        input:focus {background-color:lightgoldenrodyellow}
        input:invalid {background-color:lightgray}
        input:required {background-color:lightgreen}
    </style>
</head>
<body>
     <form>
        <input type="text" size="30" placeholder="First Name" disabled/> <br/><br/>
        <input type="text" size="30" placeholder="Last Name" autofocus/><br/><br/>
        <input type="email" size="30" placeholder="Email"/><br/><br/>
        <input type="password" size="30" placeholder="Password" required/><br/><br/>
        <input type="submit" value="sign up"  />
    </form>
</body>
</html>
```

- Lista completa de *pseudo class selectors:* http://www.w3schools.com/cssref/css_selectors.asp

# Pseudo-Elements Selectors



## Seletor CSS

- Referem-se a elementos fictícios (não correspondem a elementos HTML), os quais são baseados na estrutura do documento.
- Utilizam uma notação "**::**" diferente da pseudo-classe " **:** "
  - ***::first-line***
  - ***::first-letter***
  - ***::before***
    - Com base na propriedade ***content*** permite inserir conteúdo antes de um elemento
  - ***::after***
    - Com base na propriedade ***content*** permite inserir conteúdo depois de um elemento
  - **::selection**
    - Parte de um elemento que é selecionada pelo utilizador.

## Seletor CSS

- **::*before* *::after*** + propriedade ***content***

** ANTES ** Pseudo elements (::before::after) ** DEPOIS **

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        p  {color:gray;}
        span {color:orange;}
        p::before {content:"** ANTES **"}
        p::after  {content:"** DEPOIS **"}
    </style>
</head>
<body>
   <p><span>Pseudo elements (::before;::after)</span></p>
</body>
</html>
```



## Seletor CSS

- ***::selection***
  - permite formatar o conteúdo que está a ser selecionado pelo utilizador

Formatação diferente para uma seleção de texto

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>

    <style>
        ::selection {background-color:orange;color:white; }
    </style>

</head>
<body>
   <p><span> Formatação diferente para uma seleção de texto </span></p>
</body>
</html>
```

# Conflitos de Formatação



## CSS - Conflitos de Formatação

- Tipos de Conflito
  - Seletores iguais
    - Ordem pela qual são definidos
  - Seletores Diferentes
    - Especificidade
  - Herança

# CSS - Conflitos de Formatação

- Seletores iguais
  - Ordem pela qual são definidos
    - *last one listed wins:* **prevalece a ultima definição** para **um determinado seletor**

```
<!DOCTYPE HTML>
<html>
<head>
    <meta charset="utf-8" />
    <title> External Style Sheet</title>
    <style>
        h1{color:#F00;} /*red*/
        h1{color:#00F;} /*blue*/
    </style>
</head>

<body>
    <h1> Resolver Conflitos de Formatação </h1>
</body>
</html>
```

?

**Resolver Conflitos de Formatação**



# CSS - Conflitos de Formatação

- Seletores iguais
  - Ordem pela qual são definidos
    - Um ficheiro externo foi incorporado **depois** da definição de um *embedded style*,

```
<style> … </style>
<link …>
```

em caso de conflito **para o mesmo seletor**, prevalece a formatação estabelecida no **ficheiro externo**.

    - Um ficheiro externo foi incorporado **antes** da definição de um *embedded style*,

```
<link …>
<style> … </style>
```

em caso de conflito **para o mesmo seletor**, prevalece a formatação estabelecida no ***embedded style***.

# CSS - Conflitos de Formatação

- Conflitos
  - Seletores Iguais: Prioridade ?
    - Critério: "*Ordem pela qual são definidos*"

1. Regra assinalada como **!important** *[mais prioritário]*

```
<style>
    h1{color:#F00 !important;}
</style>
```

**!important**
Prevalece sobre todas as formatações
É a **exceção** à regra da "ordem pela qual são definidos"

2. *Inline style* (atributo **style** na *opening tag*)
3. *Embedded Style Sheet* ( ***<style>…</style>***)
4. *External Style Sheet* (***<link …/>; @import***)

Considera que o *embedded style* é declarado **após** a ligação ao ficheiro externo; caso contrário prevaleceria a declaração no ficheiro externo.

5. Definições por defeito do *browser [menos prioritário]*



# CSS - Conflitos de Formatação

- Especificidade
  - Os seletores **mais específicos tem mais peso** na resolução de conflitos de formatação:

1. ID Selectors (mais específico / mais prioritário)
2. Class Selectors; Attribute Selectors
3. Contextual Selectors (apenas com elementos)
4. Individual Element Selectors



**> especificidade**

**< especificidade**

- Entre seletores com **a mesma especificidade prevalece** a ultima declaração.

# CSS - Conflitos de Formatação

- Especificidade
  - O browser **atribui pesos específicos** de acordo com o tipo de seletor
  - Como calcular?
    - **d**: seletores de elemento e pseudo-elemento (**1 ponto**)
    - **c**: seletores de class, atributo e *pseudo-class* (**10 pontos**)
    - **b**: seletor de ID (**100 pontos**)
    - **a**: *inline style* (**1000 pontos**)

2
1

```
<style>
    body h1{color:red;}
    h1{color:blue;}
</style>
</head>

<body>
    <h1> Resolução de conflitos de formatação </h1>
```

**Resolução de conflitos de formatação**



# CSS - Conflitos de Formatação

- Conflitos de Formatação - Prioridades na aplicação de estilos
  - Especificidade
    - Como calcular?

| a | b | c | d |
|---|---|---|---|
| | | | |

13
2
1

```
    <style>
        body div.especifica h1{color:darkolivegreen}
        body h1{color:red}
        h1 {color:blue}
    </style>
</head>
<body>
    <div class="especifica">
        <h1 id="maisEspecifico">Resolução Conflitos de Formatação </h1>
    </div>
```

**Resolução Conflitos de Formatação**

# CSS - Conflitos de Formatação

- Conflitos de Formatação - Prioridades na aplicação de estilos
  - Especificidade

| a | b | c | d |
|---|---|---|---|
| | | | |

100

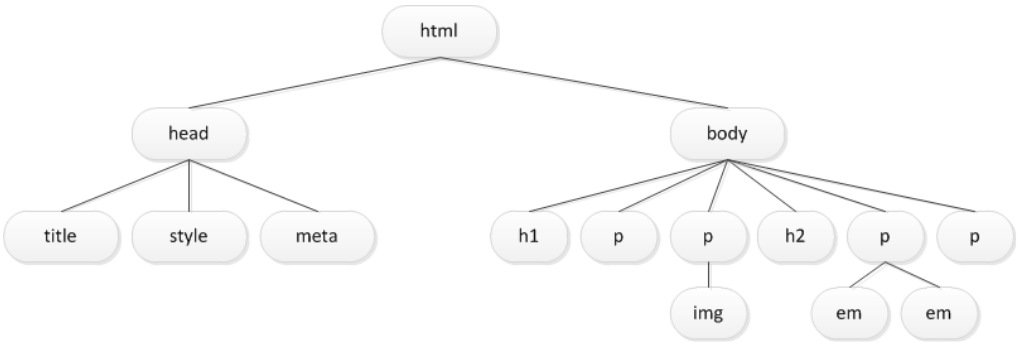
```
    <style>
        #maisEspecifico{color:darkorange}
        body div.especifica h1{color:darkolivegreen}
        body h1{color:red}
        h1 {color:blue}
    </style>
</head>
<body>
    <div class="especifica">
        <h1 id="maisEspecifico">Resolução Conflitos de Formatação </h1>
    </div>
```

**Resolução Conflitos de Formatação**



# CSS - Conflitos de Formatação

- Herança
  - Documentos HTML tem uma estrutura hierárquica implícita




- body head filhos de html
- h1 p em h2 img descendentes de body
- ...

# CSS - Conflitos de Formatação

- Herança
  - **Algumas** propriedades de **alguns** elementos HTML são herdadas
    - Geralmente as propriedades relacionadas com o estilo do texto são herdadas

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        p {color:gray;}
    </style>
</head>
<body>
   <p> Conceito Herança <span> formatação herdada de p </span></p>
</body>
</html>
```

Conceito Herança formatação herdada de p

O elemento  é formatado por herança uma vez que não foi formatado diretamente.
Na realidade, apenas foi formatado o elemento pai <p>.



# CSS - Conflitos de Formatação

- Herança
  - Só é aplicada **caso não seja definido** o estilo do elemento.

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        .p1 {color:gray;}
    </style>
</head>
<body>
   <p class="p1"> Conceito Herança <span> formatação herdada de p </span></p>
</body>
</html>
```

Conceito Herança formatação herdada de p

  - Caso o elemento seja diretamente formatado, a herança não se aplica: **prevalece sempre a formatação direta do elemento**

Conceito Herança formatação herdada de p

```
<style>
     .p1 {color:gray;}
     span {color:orange;}
</style>
```

    - Este funcionamento aplica-se sempre, mesmo que o seletor aplicado ao elemento tenha menor especificidade do que a especificidade do seletor aplicado ao elemento pai

# Unidades CSS



## Unidades CSS

- Unidades Relativas
  - São baseadas no tamanho de outro elemento/referência
    - Não pode existir um espaço em branco entre o valor e a unidade
    - Se o valor for 0 a unidade pode ser omitida
    - Por *default*, 1em corresponde a um *font-size* de 16px

*2em* ✓ *2 em* ✗

| Unit | Description |
|---|---|
| em | Relative to the font-size of the element (2em means 2 times the size of the current font) |
| ex | Relative to the x-height of the current font (rarely used) |
| ch | Relative to width of the "0" (zero) |
| rem | Relative to font-size of the root element |
| vw | Relative to 1% of the width of the viewport* |
| vh | Relative to 1% of the height of the viewport* |
| vmin | Relative to 1% of viewport's* smaller dimension |
| vmax | Relative to 1% of viewport's* larger dimension |
| % | |



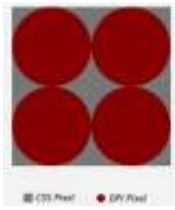
http://www.w3schools.com/cssref/css_units.asp

- *CSS pixel*
  - É uma unidade abstrata, tudo o que é definido em ***px*** em CSS baseia-se no CSS pixel
    - exemplo: {font-size:16px;}

*The **px** unit is defined **to be small but visible, and such that a horizontal 1px wide line can be displayed with sharp edges**. What is sharp, small and visible **depends on the device** and the way it is used: do you hold it close to your eyes, like a mobile phone, at arms length, like a computer monitor, or somewhere in between, like an e-book reader?*

  - O CSS pixel pode ou não coincidir com os pixéis físicos (*dpi*)
    - ***Device pixel ratio***: informação de que o browser necessita para determinar quantos pixéis físicos são utilizados para desenhar um único CSS pixel.
      - Exemplo: Os *Retina Display* da Apple possuem um *device pixel ratio* de 2

1 CSS pixel é formado por 4 pixéis físicos (2 altura/2 largura)



http://blog.popupdesign.com.br/desenvolvimento-responsivo-e-viewport/



# Box Model

Propriedades

# Box Model

- Os elementos HTML *(inline/block)* são interpretados pelo *browser* como estando contidos em caixas rectangulares, às quais podem ser aplicadas propriedades.

```
        div{
            background-color: orange;
            color:white;
            font-size: 1.5em;
            width:300px;
            height:150px;
            border:darkgray 1px solid;
        }
    </style>
</head>
<body>
    <div> Block Element </div>
```






# Box Model




- *content area*
  - *área reservada ao conteúdo*
- *padding area*
  - área definida entre a área de conteúdo e o border **(opcional)**
- *border*
  - *linha que envolve a content area e a padding área* **(opcional)**
- *margin*
  - área adicionada no exterior do *border* **(opcional)**

# CSS – Box Model

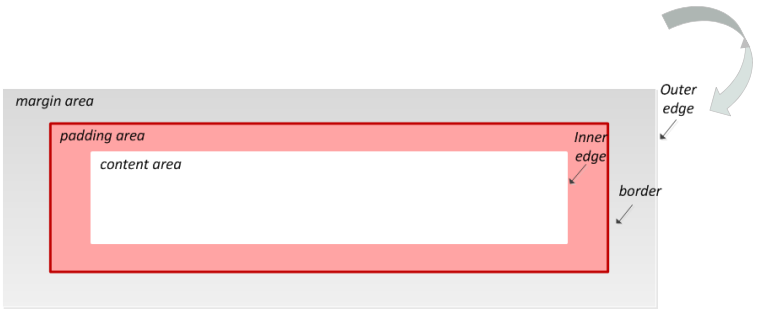
- *Propriedades*

| *Propriedade* | *Exemplo* | *Observações* |
|---|---|---|
| padding<br>*padding-top*<br>*padding-right*<br>*padding-left*<br>*padding-bottom* | p {padding:2em;} | Permite definir a *padding area*.<br>Valores: *length measurement, percentage, auto, inherit*. |
| margin<br>*margin-top*<br>*margin-right*<br>*margin-left*<br>*margin-bottom* | p {margin:2em;} | Permite definir a *margin area*.<br>Valores: *length measurement, percentage, auto, inherit*.<br><br>Nota importante: a inserção de valores negativos conduz geralmente à sobreposição dos elementos. |



# Box Model

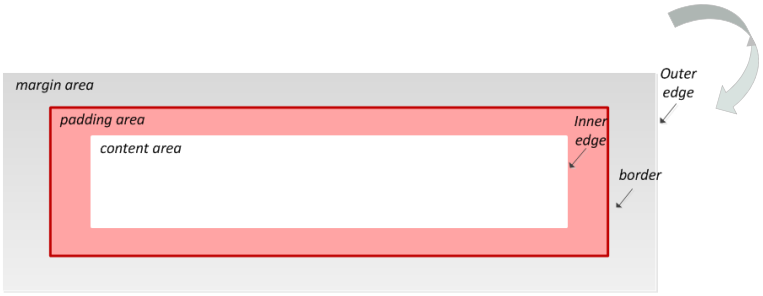
- *padding*
  - seletor{padding:5px;} 1 valor (top/bottom/right/left)
  - seletor{padding:5px 10px;} 2 valores (top/bottom right/left)
  - seletor{padding:5px 10px 15px 5px;} 4 valores (top right bottom left)




  - *padding-top; padding-right; padding-bottom; padding-left*

# Box Model

- *margin*
  - seletor{margin:5px;} 1 valor (top/bottom/right/left)
  - seletor{margin:5px 10px;} 2 valores (top/bottom right/left)
  - seletor{margin:5px 10px 15px 5px;} 4 valores (top right bottom left)




  - margin-*top;* margin-*right;* margin-*bottom;* margin-*left*



# Box Model

- *content area*
  - *width; height*
    - definem as dimensões da *content area*

```
        div{
            background-color: orange;
            color:white;
            font-size: 1.5em;
            width:300px;
            height:150px;
            border:darkgray 1px solid;
        }
    </style>
</head>
<body>
    <div> Block Element </div>
```



# Box Model

- *padding area*
  - *padding: top right bottom left;*
    - as dimensões globais resultam do somatório da área de conteúdo e de espaçamento interno

```
            div{
            background-color: orange;
            color:white;
            font-size: 1.5em;
            width:300px;
            height:150px;
            padding:20px;
            border:darkgray 1px solid;
        }
    </style>
</head>
<body>
    <div> Block Element </div>
```






# Box Model

- margin *area*
  - *margin: top right bottom left;*
    - estabelece as dimensões da *margin area* (área livre em redor do elemento)

```
        div{
            background-color: orange;
            color:white;
            font-size: 1.5em;
            width:300px;
            height:150px;
            padding:20px;
            margin:20px;
            border:darkgray 1px solid;
        }
    </style>
</head>
<body>
    <div> Block Element </div>
```

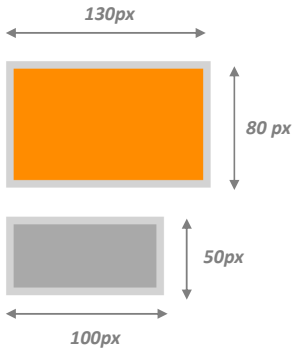


# box model

- Dimensões da área visível (***box-sizing***)
  - *content-box*
    - Valor por default, os valores de width/height referem-se à área de conteúdo
  - *border-box*
    - *content area + padding area + border*

```
#d1{width:100px;
    height:50px;
    background-color:darkorange;
    border: 5px lightgray solid;
    padding:10px; }


#d2{width:100px;
    height:50px;
    background-color:darkgray;
    border: 5px lightgray solid;
    padding:10px;
    box-sizing:border-box}
```






# Box Model

- *border*

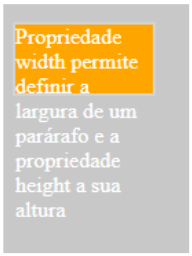
| *Propriedade* | *Exemplo* | *Observações* |
|---|---|---|
| *border-style*<br>*border-top-style*<br>*border-right-style*<br>*border-left-style*<br>*border-bottom-style* | *p {border-style: solid;}* | Permite escolher o estilo do border.<br>**Valores:** *none*, dotted, dashed, solid, double, inset (sombra interior), outset (sombra exterior) …* |
| *border-width*<br>*border-top-width*<br>*border-right-width*<br>*border-left-width*<br>*border-bottom-width* | *p {border-style: solid; border-width: 4px;}* | Permite definir a largura do border.<br>**Valores:** *length units, thin, medium*, thick, inherit* |
| *border-color*<br>*border-top-color*<br>*border-right-color*<br>*border-left-color*<br>*border-bottom-color* | *p {border-style: solid; border-width: 4px; border-color: red;}* | Definição da cor do border.<br>**Valores:** *color name/RGB value, transparent, inherit* |
| *border*<br>*border-top*<br>*border-right*<br>*border-left*<br>*border-bottom* | *p{border: 4px solid red; }* | Propriedade agregada (width, style, color)<br>Mais frequentemente utilizada. |

# Box Model

- *overflow*

| *Propriedade* | *Exemplo* | *Observações* |
|---|---|---|
| overflow | p{overflow:scroll;} | Valores: *visible, hidden, scroll, …* |

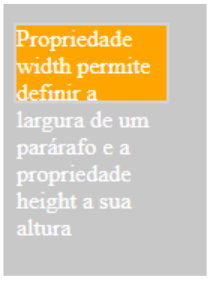
```
        body{background-color:rgb(200,200,200)}
            p{
                height:50px;
                width:100px;
                background-color: orange;
                border:2px lightgray solid;
                color:white
            }
    </style>
</head>
<body>
    <p> Propriedade width permite definir a largura de um parárafo e a
        propriedade height a sua altura</p>
```






# CSS – Box Model

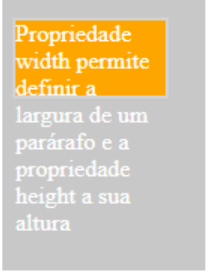
```
p{
  height:50px;
  width:100px;
  background-color: orange;
  border:2px lightgray solid;
  color:white;
  overflow:hidden}
```







```
p{
  height:50px;
  width:100px;
  background-color: orange;
  border:2px lightgray solid;
  color:white;
  overflow:scroll}
```






# box model

- Cantos arredondados
  - border-radius: valor;
    - O mesmo valor aplica-se a todos os cantos
  - border-radius: valor1 valor2;
    - valor1: canto superior direito/inferior esquerdo
    - valor2: canto inferior direito/superior esquerdo
  - border-radius: valor1 valor2 valor3 valor4;
    - valor1 canto superior esquerdo (sentido horário)
  - Propriedades individuais:
    - border-top-left-radius
    - border-top-right-radius
    - border-bottom-right-radius
    - border-bottom-left-radius

```
<!DOCTYPE html>
<html>
<head>
    <style>
    p{background-color:darkorange;
      color:white;
      width:200px;
      border:darkorange 2px solid;
      border-radius:4px;}
    </style>
</head>
<body>
    <p>BORDER RADIUS</p>
</body>
</html>
```

BORDER RADIUS



# box model

- Efeito Sombra (box)
  - {*box-shadow: h-shadow v-shadow blur (opt.) spread (opt.) color (opt.) inset (opt.);*}

```
<!DOCTYPE html>
<html>
<head>
     <style>
        #d1 {width:100px;height:50px;
             background-color:darkorange;
             color:white;
             border:5px lightgray solid;
             padding:10px;
             box-shadow:lightgray 10px 10px 5px;}
    </style>
</head>
<body>
    <div id="d1">Box Shadow</div><br/>
</body>
</html
```

Box Shadow

# Propriedades

*display*